TraceIQ
Product Requirements and Build Plan

Evidence-grounded incident investigator • MongoDB Atlas RAG Engine (Track 2)
Version 2.0 • 24 October 2026 • Owner: Parth Patidar
Matches main at commit 344cfad plus PR 'fix/demo-blockers'

How to use this document

- **Everyone** reads sections 1 to 3 and does the setup in section 7 first.
- **Junior A** then works through task cards A1 to A6 in section 9. **Junior B** works through B1 to B6. The two lists never edit the same files, so you will not block each other.
- The code on main already works end to end. Your tasks improve it; they do not rewrite it. Keep existing names, files and patterns. Do not add a library, collection, service or file that a task card does not list. If you think one is needed, ask Parth first.
- One branch and one pull request per task card. A task is done only when every 'Done when' line passes and the output is pasted into the PR.
- If anything here is unclear or looks wrong, stop and ask. Do not guess, and do not invent data, page numbers or API behaviour.

Contents

- 1. Product: what TraceIQ is and how it meets the brief
- 2. Technology stack and decisions
- 3. How the system works today (with file names)
- 4. Repository map and who changes what
- 5. Data model and search indexes (as built)
- 6. API (as built, plus additions from the tasks)
- 7. Setup guide (step by step)
- 8. Testing and evaluation
- 9. Work plan: task cards for Junior A and Junior B
- 10. Reference facts for the tasks (measured on the demo data)
- Appendix A. The 20 test questions • Appendix B. True PDF page numbers • Appendix C. Glossary

1. Product: what TraceIQ is and how it meets the brief

1.1 The problem, from the user's side

Before. It is 10:15 on a Friday. Checkout payments are failing. Priya, the on-call engineer, opens the logs, the metrics dashboard, the API spec PDF, the release notes, the dependency's changelog and the recording of the incident call, each in a different tool. Her first report guesses "expired credentials" because the error code is PAY-401. It takes the team an hour to learn the code means a validation failure, not an authentication failure.

With TraceIQ. All of that evidence sits in one incident. Priya asks "Why did payments start failing after the deploy?" TraceIQ returns a timeline and the most likely cause, marked as an inference, with a citation on every claim: log LOG-0009, page 7 of the API spec, cue 4 of the incident call, the schemaguard release notes. Asked "Who approved the deploy?", it says the evidence does not show this.

1.2 What TraceIQ does and does not do

- Evidence is grouped by **incident** (incident_id). An incident is our 'project': files are uploaded once, processed once and kept; new files can be added any time; every search stays inside one incident.
- Accepts documents (PDF, DOCX, MD), webpages (HTML), images (PNG, JPG, WEBP), transcripts (VTT, TXT) and structured records (JSON, CSV).
- Answers with evidence from several sources: claims marked confirmed, inference or unknown; each claim independently checked against its cited text; a timeline; conflicting evidence; recommended checks.
- Says the evidence is insufficient (abstains) instead of guessing.

Out of scope: logins, chat history, audio and video, scanned PDFs (no OCR for PDFs), rerankers, Temporal, cloud deployment.

1.3 The brief, line by line

The brief asks	Today on main	Finished by
A focused collection of docs, webpages, images, transcripts, records	Done: demo pack, 20 files, all 5 types	-
Retrieve from more than one source or section	Done: hybrid search + query variants	-
Show the source behind every important result	Done: claims cite chunk IDs; citations show location	A1-A3 (exact sections), B3 (open original)
Handle a difficult or unanswerable request	Done: verifier + abstention	B6 (no hard failure)
Demo: import a collection and make it searchable	Done: Import button and upload	B4 (pick incident)
Demo: open citations and show exact passages	Partly: passage shown, no original file	B1, B3
Store sources, chunks, embeddings, metadata, IDs together	Partly: uploads saved on disk	B1 (GridFS)
MongoDB Vector Search	Done: $vectorSearch, 768 dims	-
Filters for dates, owners, content types	Types work; owner is empty; dates stored as text	A5, A6
Hybrid search for exact names and codes	Done: vector + BM25 + exact identifiers, fused	-
Test requests with expected source IDs	Done: /api/eval/benchmark on the 20 questions	-
Multimodal (images)	Partly: images indexed by manifest text only	A4 (Gemini reads images)

1.4 The demo (about 5 minutes)

1. **Import.** Pick incident INC-2026-1042, click Import / refresh dataset, watch the sources appear with chunk counts. Upload one extra screenshot.
2. **Cross-source question** (Q05): 'Why did payment requests start failing after the latest deployment? Reconstruct the timeline, identify the most likely cause, explain the evidence, and recommend the next diagnostic step.'

3. **Open citations.** Click a citation: the modal shows the passage and its location; Open original shows the PDF page or the screenshot (after B1 and B3).
4. **Hard requests.** Q15 'The initial report blames a credential problem. Is that still the best explanation?' (conflict) and Q17 'Which person approved the payment-service 2.4.0 production deploy?' (must abstain). These four questions are the demo buttons in the UI.

2. Technology stack and decisions

2.1 Stack (as on main)

Part	Technology	Where
Frontend	Next.js 16, React 19, TypeScript, Tailwind CSS 4	frontend/src
Backend	Python 3.12, FastAPI, Pydantic v2, pydantic-settings	backend/app
Database	MongoDB Atlas (team demo) or MongoDB Atlas Local in Docker (each laptop); pymongo	core/database.py
Search	Atlas Vector Search ($vectorSearch, 768 dims, cosine) + Atlas Search BM25 ($search) + exact identifier lookup; fused with reciprocal rank fusion in Python	retrieval/
AI	Gemini via google-genai: gemini-embedding-001 (768 dims) and gemini-2.5-flash	services/gemini.py, embeddings.py
Parsing	PyMuPDF (PDF), python-docx, BeautifulSoup (HTML), json/csv, own VTT/TXT/MD	ingestion/parsers.py
Chunking	langchain-text-splitters with tiktoken: 500 tokens, 60 overlap	ingestion/chunking.py
Tests	unittest classes run by pytest (19 tests on main, 24 after fix/demo-blockers)	backend/tests

2.2 Decisions

#	Decision	Why
D1	The incident in synthetic_data (INC-2026-1042) and its 20 eval questions are the test set	They exist and agree with each other. backend/tests/ground_truth.json points to them.
D2	Gemini (free tier) for embeddings and answers	Free; one key
D3	768-dimension embeddings	Already built and indexed; half the storage of 1536 on the 512 MB free cluster
D4	incident_id is the project	Every query already filters on it; no new collection needed
D5	Original files in MongoDB GridFS (task B1)	The brief says store originals with chunks and embeddings
D6	Fusion stays in Python (reciprocal_rank_fusion)	Works on every MongoDB version; three lists (vector, BM25, identifiers)
D7	Each junior develops on MongoDB Atlas Local in Docker	The free Atlas cluster allows only 3 search indexes in total; each database needs 2
D8	No new collections, services or frameworks in the tasks	Small team, small codebase; extend what exists

3. How the system works today (with file names)

3.1 Ingest

1. **Import dataset**: POST /api/ingest/batch, routes_ingest.py, calls services/ingestion.py ingest_dataset(). It reads synthetic_data/metadata/source_manifest.json (never metadata/ or evaluation/ as evidence).
2. **Upload**: POST /api/ingest/upload, one file with source_type, incident_id and owner. ingest_uploaded_file() checks type and size, saves the file to backend/uploads/ (B1 moves this to GridFS), and builds a manifest-like entry with source_id 'UPL-...'.
3. **Index one source**: _index_source() calls parsers.parse_source() which returns units (text + locator), then chunking.chunk_evidence_units() which splits units into chunks of 500 tokens and adds identifiers, incident_id, owner, version and timestamp.
4. If every chunk already exists with the same source_hash, the source is skipped. Otherwise new chunks are embedded (embeddings.embed_texts), upserted by chunk_id, and chunks that no longer exist are deleted.
5. Each run is recorded in the ingestion_runs collection (GET /api/ingest/runs/{run_id}).

3.2 Ask

1. POST /api/investigate/ (routes_investigate.py) with query and optional incident_id, owner, version, source_types and time range.
2. query_transform.decompose_query() rewrites abbreviations and may split the question into up to 3 variants (regex, no AI call).
3. For each variant, hybrid_search.perform_hybrid_search() runs three searches and fuses them: $vectorSearch (meaning), $search BM25 over text, title and identifiers (words), and an exact identifier lookup (codes like PAY-401, trc-5a10c4, versions).
4. investigator.select_evidence() keeps at most 3 chunks per source and 10 in total.
5. Gemini writes the answer as JSON (claims with chunk IDs, conflicts, actions, uncertainties). verify_claims() asks Gemini a second time whether each claim is supported by its cited text; unsupported claims become 'unknown'. No verified claim means abstention.

Rule that must never be broken: every search and list filters on incident_id when one is given. The frontend always sends one. A question must never see evidence from another incident.

4. Repository map and who changes what

File	Job	Changed by
backend/app/config.py	All settings (.env)	A5, A6 add settings
backend/app/core/database.py	Mongo client, regular indexes, search index readiness	B1
backend/app/ingestion/parsers.py	File to units with locators	A1, A2, A3, A4, A5
backend/app/ingestion/chunking.py	Units to chunks, identifiers, owner, timestamp	A5, A6
backend/app/services/gemini.py	Gemini client and JSON generation	A4
backend/app/services/embeddings.py	Gemini embeddings	nobody
backend/app/services/ingestion.py	Import dataset, upload, index one source	B1, B2
backend/app/retrieval/vector_search.py	$vectorSearch and filters	A6
backend/app/retrieval/hybrid_search.py	BM25, identifier lookup, fusion	A6
backend/app/reasoning/investigator.py	Evidence selection, answer, verification, abstention	B6
backend/app/reasoning/query_transform.py	Query rewrite and split	nobody
backend/app/api/routes_ingest.py	Ingest routes	B1, B4, B5
backend/app/api/routes_investigate.py	Investigate route	B5
backend/app/api/routes_eval.py	Benchmark	nobody
backend/scripts/manage_vector_index.py and the two index JSON files	Create and check search indexes	A6
backend/tests/test_rag_core.py	All tests	both (add tests in your own test class)
frontend/src/app/page.tsx	Main page, API calls, filters	B4
frontend/src/components/CitationModal.tsx	Citation detail	B3
frontend/src/lib/demoQuestions.ts	The 4 demo questions	nobody

If two tasks must touch the same file (tests, config.py), keep your change in its own block and merge main into your branch before opening the PR.

5. Data model and search indexes (as built)

Database traceiq_db. Collections: sources, chunks, ingestion_runs. Task B1 adds the GridFS bucket 'originals' (originals.files, originals.chunks).

5.1 sources (one per file)

Manifest fields (source_id, title, source_type, format, file_path, created_at, published_at, version, incident_id, services, tags, description, sha256, recording_start, canonical_url) plus source_hash, size_bytes, analysis_status, ingestion_status, chunk_count, updated_at. Unique index on source_id. Tasks add: owner (A5), file_id (B1).

5.2 chunks (one per searchable passage)

```
{
  "chunk_id": "uuid5(...)",            // unique; stable while source_hash, locator and text are unchanged
  "source_id": "DOC-002", "source_type": "document", "title": "Payment API Specification v2.0",
  "text": "...", "content_hash": "sha256 of text", "chunk_index": 0, "start_index": 0,
  "locator": {"page": 7},              // A1-A3 add section / html_id; records keep log_id, row_id, ...
  "incident_id": "INC-2026-1042", "owner": null, "version": "2.0",
  "timestamp": "2026-10-23T09:41:02.118Z",   // text today; A6 stores a real date
  "source_hash": "...",
  "metadata": {"services": [...], "tags": [...], "record": {...}, "speaker": "...", ...},
  "identifiers": {"error_code": ["pay-401"], "log_id": ["log-0009"], ...},  // lowercase
  "embedding": [768 floats]
}
```

5.3 Search indexes (backend/scripts/*.json, created by manage_vector_index)

- **traceiq_vector_index** (vectorSearch): embedding 768 cosine; filters incident_id, source_type, version, timestamp, owner.
- **traceiq_search_index** (search): text and title (lucene.standard), incident_id, source_type, version, owner (token), timestamp (string today; date after A6), identifiers (document, dynamic).
- Changing an index definition means deleting the index and running the create script again, then re-importing (A6 is the only task that does this).

6. API (as built, plus additions from the tasks)

Method and path	What it does
GET /health, GET /ready	Liveness; readiness of MongoDB, both indexes, Gemini key, dataset
POST /api/ingest/batch?incident_id=	Import or refresh the demo dataset (repeat-safe)
POST /api/ingest/upload	Multipart: file, source_type, incident_id, owner. One file per call
GET /api/ingest/sources?incident_id=	List sources
GET /api/ingest/runs/{run_id}	One ingestion run summary
POST /api/investigate/	Ask a question (InvestigationRequest) and get an InvestigationReport
GET /api/eval/benchmark?top_k=10&generate_answers=false&question_ids=Q05	Retrieval metrics on the 20 questions; with generate_answers=true also answer checks
NEW (B1) GET /api/ingest/sources/{source_id}/original	The original file bytes
NEW (B4) GET /api/ingest/incidents	Distinct incident_id values with source counts

- Errors: 400 bad input, 422 validation, 503 dependency problems (MongoDB, Gemini, indexes still building). Task B5 adds 429 for Gemini quota.
- The frontend reads the backend URL from NEXT_PUBLIC_API_BASE_URL (default http://localhost:8000).

7. Setup guide (step by step)

7.1 Install once

- Git, Python 3.12, Node.js 20+, Docker Desktop, MongoDB Compass (optional, to look at data).
- Clone: `git clone https://github.com/CodingTJ1507/RAG_Transfinitte.git`, then `git pull` before starting each task.

7.2 MongoDB for development: Atlas Local in Docker (each junior)

```
docker run -d --name traceiq-mongo -p 27017:27017 -v traceiq-db:/data/db -v traceiq-config:/data/configdb mongodb/mongodb-atlas-l
```

- In backend/.env: `MONGODB_URI=mongodb://localhost:27017/?directConnection=true`.
- The -v volumes keep your data when the container restarts. Next time just run `docker start traceiq-mongo`.

7.3 MongoDB for the team demo: Atlas free cluster (Parth)

1. cloud.mongodb.com: create project TraceIQ, create a Free cluster near you.
2. Database Access: add a user. Network Access: Add Current IP Address (each machine that connects).
3. Connect, Drivers, Python: copy the mongodb+srv string into MONGODB_URI (URL-encode special characters in the password).
4. Only the demo database lives here. The free cluster allows 3 search indexes in total.

7.4 Gemini key

- https://aistudio.google.com/apikey: Create API key, paste into GEMINI_API_KEY in backend/.env. Each junior uses their own key (separate free quota).
- Free tier limits are per key; see https://aistudio.google.com/rate-limit. Google may use free-tier prompts to improve its products: use only the synthetic demo data.

7.5 Backend

```
cd backend
py -3.12 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
Copy-Item .env.example .env          # then set MONGODB_URI and GEMINI_API_KEY
python -m scripts.manage_vector_index --create
uvicorn app.main:app --reload --port 8000
```

- Run scripts with `python -m scripts.name` from backend/. `python scripts\name.py` fails with 'No module named app'.
- Open http://localhost:8000/ready: database, vector_search and atlas_search should all be ready.
- Import the demo: `Invoke-RestMethod -Method Post http://localhost:8000/api/ingest/batch` (or the Import button in the UI).

7.6 Frontend

```
cd frontend
npm ci
npm run dev                          # http://localhost:3000
```

7.7 Troubleshooting

Symptom	Fix
SSL alert number 80 / TLSV1_ALERT_INTERNAL_ERROR	Your IP is not in Atlas Network Access. Add it. Else try a phone hotspot (network blocks port 27017), turn off VPN or HTTPS-scanning antivirus, check the cluster is not paused.
No module named 'app'	Run scripts as python -m scripts.name from backend/.
503 'Search indexes are still building'	Wait a minute and retry, or run python -m scripts.manage_vector_index again (it waits up to SEARCH_INDEX_TIMEOUT_SECONDS, default 90 s).
'Vector index uses N dimensions'	Index and EMBEDDING_DIMENSIONS differ. Delete the index, run the create script, re-import.
Too many search indexes on Atlas	Use Docker for development (7.2); the free cluster holds one database's indexes.
503 from investigate after many questions	Gemini quota. Wait a minute (B5 turns this into a clear 429).
Data gone after Docker restart	Container started without the -v volumes (7.2).

8. Testing and evaluation

8.1 Every PR

```
cd backend
python -m pytest -q                                  # must stay green (24 on main after fix/demo-blockers)
python -m scripts.validate_synthetic_data synthetic_data
cd ../frontend
npm run lint
npm run build
```

- **Test first.** Write the new test, run it, and paste the failure in the PR. Then fix the code and paste the pass. A test that never failed proves nothing.
- Put your tests in backend/tests/test_rag_core.py in a new class named after your task (e.g. class HtmlSectionTests). No real MongoDB or Gemini calls in tests: mock them with unittest.mock.patch, as the existing tests do.

8.2 Retrieval benchmark (A tasks, before and after)

```
Invoke-RestMethod 'http://localhost:8000/api/eval/benchmark?top_k=10' | Select-Object -ExpandProperty aggregate
Invoke-RestMethod 'http://localhost:8000/api/eval/benchmark?top_k=10&question_ids=Q01&question_ids=Q05'
```

- Reports hit rate, recall, precision, MRR and nDCG per question against synthetic_data/evaluation/expected_sources.json. Uses one Gemini embedding call per question.
- Every A task pastes the aggregate numbers before and after its change. A change that lowers mean recall needs a written reason in the PR.
- generate_answers=true also runs the answer and verifier (3 Gemini calls per question); use it with question_ids to save free quota.

9. Work plan: task cards

9.1 Rules for both juniors

1. git pull on main, then create a branch: feat/a1-html-sections, feat/b1-gridfs-originals, and so on.
2. Touch only the files listed on your card. If you need another file, ask Parth first.
3. Follow the PR template (.github/pull_request_template.md): what changed, what you learned, how you tested (real output), screenshots for UI work.
4. Keep functions small and names consistent with the file you are in. No new libraries.
5. Order: Junior A does A1, A2, A3, A5, A4, A6. Junior B does B2, B1, B3, B4, B5, B6. A5 needs A3 merged (cover owner). B3 needs B1 merged. A4 is cheaper after B2 (fewer Gemini calls on re-import).

9.2 Junior A: ingestion quality

Area: how files become chunks. Backend only. Your files: ingestion/parsers.py, ingestion/chunking.py, services/gemini.py, retrieval/vector_search.py, retrieval/hybrid_search.py, config.py (new settings only), scripts/atlas_search_index.json, scripts/manage_vector_index.py.

A1. HTML: one chunk per section

Why: Today every paragraph is its own chunk and 9 of 13 lose their section id (some chunks are 8 characters). Small chunks search badly and cite vaguely.

Files: ingestion/parsers.py (_parse_html), tests

Build:

- Walk the main element in order. Each h2 starts a new unit; collect the text of following p, li, pre, blockquote and table elements (and h3/h4 headings) into it until the next h2.
- Text between the h1 and the first h2 is one unit with section = the h1 text and html_id 'intro'.
- locator = {"section": h2 text, "html_id": the h2 id, or a slug of its text if it has none}.

Done when:

- WEB-001 (dependency_release_notes.html) gives exactly the html_ids intro, overview, breaking-changes, migration, 3-0-1 in that order.
- WEB-002 gives 4 non-intro units and WEB-003 gives 6 (one per h2).
- Benchmark before and after pasted.

A2. TXT: one chunk per section

Why: The handover note (TRN-003) becomes 32 one-line chunks, some 6 characters long, and its title line is treated as a section.

Files: ingestion/parsers.py (_parse_text), tests

Build:

- The first non-empty line is the title, never a section.
- After it, a line that is all capitals and under 40 characters starts a section. Lines between the title and the first section form section 'HEADER'.
- One unit per section: its lines joined. locator {"section": name, "line": first line number}.
- A TXT with no capital section lines: units of consecutive lines up to about 2,000 characters, split at blank lines; section 'Part 1', 'Part 2', ...

Done when:

- TRN-003 gives exactly 5 units with sections HEADER, STATUS, VERIFIED, NOT VERIFIED / STILL OPEN, NEXT STEPS.
- A test TXT with no capital lines, made of five 1,000-character paragraphs separated by blank lines, gives 3 parts (2 + 2 + 1 paragraphs).
- Benchmark before and after pasted.

A3. PDF: page and section in every citation

Why: PDFs are split by page only, so a citation says 'Page 7' and the eval's section locators can never match.

Files: ingestion/parsers.py (_parse_pdf), tests

Build:

- Use page.get_text('dict') and read each line's spans (text, font, size).
- A line is a heading when every non-space span uses a font whose name contains 'Bold' and round(size) is 16 (section) or 12 (subsection). See 10.1 for the measured fonts.
- section_path: a size-16 heading resets it to [heading]; a size-12 heading makes it [section, heading]. It carries over to the next page until a new heading.
- Skip the contents page (a line 'Contents' plus 3 or more lines containing ' . . ').
- Drop every line containing 'Page N of M' (the footer, 10.1).
- Page 1 is the cover: one unit, section 'Cover'. Read the 'Owner' label (bold span) and its value (next span) and return it as cover_owner in the parsed source (A5 uses it).
- One unit per page per section_path. locator {"page": n, "section": last item, "section_path": [...]}. citation_from_document already prints 'Page 7, Section ...'.

Done when:

- DOC-002 has a unit on page 7 whose section_path contains '4. Deprecated Fields' and one on page 8 with '5. Error Codes'.
- DOC-004 has '2.1 Dependency Updates' on page 4 and '4. Compatibility' on page 5. DOC-001 has '4. Retry Behaviour Between Services' on page 7.
- Cover owners: DOC-001 Platform Team, DOC-002 Payments Team, DOC-003 SRE Team, DOC-004 Payments Team, DOC-005 none (it has 'Author').
- No unit text contains 'Page 3 of' or ' . . '. Benchmark before and after pasted.

A5. Owner on every source and chunk

Why: Owner is empty everywhere, so the brief's 'filter by owner' does nothing.

Files: ingestion/parsers.py, ingestion/chunking.py, config.py, tests

Build:

- config.py: SERVICE_OWNERS, a JSON string setting, default '{"payment-service": "Payments Team", "order-service": "Orders Team"}' (the two teams named in the handover). Add a property that returns it as a dict. Add it to .env.example.
- Source owner, first match: owner given in the manifest or upload; the PDF cover_owner (A3); SERVICE_OWNERS[first service of the source]; else 'Unassigned'.
- Chunk owner: for a record, SERVICE_OWNERS of the record's own service or client_service, else 'Unassigned'; otherwise the source owner. Spell values exactly as above.

Done when:

- After re-import: DOC-001 Platform Team, DOC-002 Payments Team, DOC-003 SRE Team, DOC-005 Payments Team (its first service is payment-service).
- REC-001: LOG-0001 (api-gateway) is Unassigned; LOG-0002 (payment-service) is Payments Team.
- POST /api/investigate/ with owner 'Payments Team' cites only Payments Team chunks (paste output).

A4. Images: let Gemini read them

Why: Images are indexed only by the manifest's description. An uploaded screenshot is indexed as just its filename, so the demo's 'upload a screenshot' step finds nothing.

Files: services/gemini.py (new function), ingestion/parsers.py (image branch), tests

Build:

- gemini.py: describe_image(data: bytes, mime_type: str) -> dict. Call generate_content with types.Part.from_bytes(data=data, mime_type=mime_type) and the prompt below, response_mime_type

'application/json'. Return {description, visible_text}. Raise RuntimeError on failure, like generate_json.

- Image unit text: title, the manifest description if any, the Gemini description, then 'Visible text:' and every line. locator {"image": name, "analysis": "gemini_vision"}; analysis_status 'vision_analyzed'.
- Prompt: 'You are reading an image uploaded as incident evidence. Return JSON with description (3 to 6 sentences: every visible number, time, version, error code, label and trend; do not guess anything not visible) and visible_text (every readable line exactly as written, top to bottom).'

Done when:

- Unit test with describe_image mocked: the chunk contains the visible text.
- After a real import, the IMG-002 chunk contains 'trc-5a10c4' and 'HTTP/1.1 422' (paste it).
- Benchmark for Q01 retrieves IMG-002.

A6. Store timestamps as real dates

Why: Timestamps are stored as text. Date range filters need real dates to work reliably in both indexes.

Files: chunking.py, vector_search.py, hybrid_search.py, scripts/atlas_search_index.json, scripts/manage_vector_index.py, tests

Build:

- chunking.py: store timestamp as a timezone-aware UTC datetime (parse the ISO text); None if missing or unparsable.
- Filters: pass datetime values (not strings) in build_vector_filter and _atlas_search_filters.
- Search index: timestamp becomes {"type": "date"} in both the JSON file and manage_vector_index.py.
- citation_from_document must still return metadata timestamp as an ISO string ending in 'Z' (the timeline in investigator.py needs a string).
- Migration on your Docker database: delete traceiq_search_index (Compass or Atlas UI), delete the chunks collection, run python -m scripts.manage_vector_index --create, re-import. Write these steps in the PR for Parth.

Done when:

- Unit test: build_vector_filter with a time range produces datetime values.
- After re-import, a chunk's timestamp is a date in Compass, not a string.
- Investigate with time_range_start 2026-10-23T10:00:00Z and time_range_end 2026-10-23T10:10:00Z returns only evidence inside that window (paste the timestamps).

9.3 Junior B: storage, API and frontend

Area: where files live, the API around them and the screens. Your files: services/ingestion.py, core/database.py, api/routes_ingest.py, api/routes_investigate.py, a new api/errors.py (B5 only), reasoning/investigator.py, frontend/src/app/page.tsx, frontend/src/components/CitationModal.tsx, frontend/src/lib/types.ts.

B2. Skip unchanged files before parsing

Why: Today every import parses every file again, even unchanged ones. After A4 that would call Gemini for every image on every import and waste free quota.

Files: services/ingestion.py (_index_source), tests

Build:

- Before calling parse_source, read the file bytes, compute the hash the same way parsers._source_hash does (import it, do not copy it), and look up the sources record.
- If the record has the same source_hash, ingestion_status 'indexed' and chunk_count above 0, return 'skipped' without parsing.

Done when:

- Test: with parse_source patched to raise, re-importing an unchanged source returns 'skipped' and parse_source is never called.
- A changed file is still parsed and re-indexed.

B1. Keep original files in MongoDB (GridFS)

Why: Uploads are saved on the laptop's disk, not in MongoDB. The brief says store originals together with chunks and embeddings, and a citation cannot open its file.

Files: core/database.py, services/ingestion.py, api/routes_ingest.py, tests

Build:

- database.py: get_originals_bucket() returns gridfs.GridFS(get_db(), collection='originals').
- When a source is indexed (dataset or upload), store its bytes once: look up originals.files by metadata.sha256; if absent, put the bytes with metadata {source_id, incident_id, filename, content_type, sha256}. Save the id as file_id on the sources record.
- backend/uploads/ stays only as the local copy the parser reads; GridFS is the stored original.
- New route GET /api/ingest/sources/{source_id}/original: 404 if unknown; else the bytes with Content-Type application/pdf or image/* for those formats and text/plain for html, json, csv, vtt, txt and md (uploaded HTML must never run), plus X-Content-Type-Options: nosniff.

Done when:

- After importing the demo, Compass shows 20 documents in originals.files.
- Upload a PNG, then GET its original: the sha256 of the response equals the file's (paste both).
- Unit test for the route with the bucket mocked: content type and 404.

B3. 'Open original' in the citation modal

Why: Users should see the real page or screenshot behind a citation, not only the extracted text.

Files: frontend/src/components/CitationModal.tsx

Build:

- Add an 'Open original' button. On click, fetch the B1 route. On success, make a blob URL and open it in a new tab, adding '#page=N' for PDFs when the citation locator has a page.
- On failure, show 'Original file unavailable right now' inside the modal. Never a blank screen.

Done when:

- npm run lint and npm run build pass.
- Screenshots in the PR: a DOC-002 citation opening the PDF at its page; an image opening; the error message with the backend stopped.

B4. Pick or create an incident (the project selector)

Why: The UI hard-codes INC-2026-1042 in 4 places, so a user cannot keep a second set of evidence.

Files: api/routes_ingest.py, frontend/src/app/page.tsx, frontend/src/lib/types.ts

Build:

- Backend: GET /api/ingest/incidents returns [{incident_id, source_count}] from sources (one aggregate), sorted by incident_id.
- Frontend: selectedIncident state, remembered in localStorage, default INC-2026-1042. Replace every hard-coded 'INC-2026-1042' with it. The sidebar lists incidents and has 'New incident' (asks for an ID, upper-case letters, digits and '-', 3 to 40 characters).
- 'Import / refresh dataset' only shows for INC-2026-1042 (the dataset's incident).

Done when:

- Create INC-TEST-1, upload one TXT, ask about it: citations come only from that file.
- Switch back to INC-2026-1042: the demo sources and answers are unchanged.
- lint and build pass; screenshots in the PR.

B5. Tell the user when the AI quota is used up

Why: A Gemini quota error today shows as 'service unavailable' (503), so users do not know to wait and retry. The frontend already shows 'Service Busy' for 429.

Files: new api/errors.py (one helper), api/routes_investigate.py, api/routes_ingest.py, tests

Build:

- errors.py: is_quota_error(exc) walks exc.__cause__ / __context__ and returns True if any is a google.genai.errors.APIError with code 429.
- In the RuntimeError handlers: if is_quota_error, raise HTTPException 429 'AI quota reached. Try again in a minute.'; otherwise keep 503.

Done when:

- Test: a RuntimeError raised from a genai ClientError with code 429 gives 429; any other RuntimeError still gives 503.

B6. Answer even if the checker fails

Why: If Gemini's verification step returns anything slightly off, verify_claims raises and the whole answer fails with 503.

Files: reasoning/investigator.py, tests

Build:

- In investigate_query, wrap the verify_claims call. On RuntimeError keep the claims, set verification_status 'not_checked', treat cited non-unknown claims as the supported set, add the uncertainty 'Claims could not be automatically verified.' and set confidence_explanation to say so.
- Do not change verify_claims itself.

Done when:

- Test: with verify_claims patched to raise, the report is not an abstention, its claims are 'not_checked' and the uncertainty is present.
- The existing verifier tests still pass.

10. Reference facts for the tasks (measured on the demo data)

10.1 PDFs

- All 5 PDFs: section headings are Helvetica-Bold size 16, subsections Helvetica-Bold size 12. Bold lines of size 8 to 10 are table headers or labels.
- Page 1 is a cover with 'Label Value' lines where the label is bold: e.g. 'Owner Platform Team' (DOC-001), 'Owner SRE Team' (DOC-003), 'Owner Payments Team' (DOC-002, DOC-004). DOC-005 has 'Author Priya Nair, ...' and no Owner.
- Page 2 is a contents page with dot leaders.
- Footer line on every page, e.g. 'SS-ARCH-001 | Version 1.3 | FICTIONAL / SYNTHETIC DATA Page 3 of 11'. The page number changes, so match the 'Page N of M' pattern.

10.2 Other files

Source	Fact
WEB-001	h2 ids: overview, breaking-changes, migration, 3-0-1 (plus intro text after the h1)
WEB-002 / WEB-003	4 and 6 h2 sections
TRN-001 / TRN-002	12 and 18 cues; recording start 09:30:00Z and 10:20:00Z; speakers in tags
TRN-003	Title line, then HEADER lines, then STATUS, VERIFIED, NOT VERIFIED / STILL OPEN, NEXT STEPS
REC-001	30 log records (log_id); REC-002 60 metric rows (row_id); REC-003 8 events (event_id)
REC-004	19 payment request records (record_id); REC-005 6 services + 8 dependencies
Demo pack	21 files, 0.70 MB (20 evidence files + webpages/sources.json index)

Appendix A. The 20 test questions

From synthetic_data/evaluation/questions.json. Demo buttons: Q05, Q06, Q15, Q17.

ID	Category	Question	Required sources
Q01	single source	What error code and HTTP status were returned for request req-1107, and what was its trace ID?	IMG-002
Q02	single source	How many times does order-service retry a failed payment call, and what backoff does it use?	DOC-001
Q03	single source	According to the Payment API Specification, until when is the card_token field accepted?	DOC-002
Q04	single source	Which validation library version change is included in payment-service 2.4.0?	DOC-004
Q05	cross source	Why did payment requests start failing after the latest deployment? Reconstruct the timeline, identify the most likely cause, explain the evidence, and recommend the next diagnostic step.	REC-003, REC-001, REC-004, DOC-004, WEB-001, DOC-002, TRN-003
Q06	cross source	Is PAY-401 an authentication error?	DOC-002, REC-001, IMG-002, WEB-003
Q07	cross source	Why did some payments still succeed while payment-service 2.4.0 was running?	REC-004, DOC-001, TRN-001
Q08	cross source	Why did order-service payment latency rise to about 1.5 seconds while payment-service latency went down?	REC-002, DOC-001, REC-001, TRN-002
Q09	cross source	Did Redis or database slowness cause the payment failures?	TRN-002, REC-001, REC-002
Q10	cross source	Was rolling back to 2.3.1 safe according to the runbook, and did it work?	DOC-003, DOC-004, REC-003, REC-002
Q11	timeline	Reconstruct the incident timeline from deploy start to recovery.	REC-003, REC-001, REC-002, IMG-004
Q12	timeline	How long after the deploy completed did the first PAY-401 appear, and how long after that did the alert fire?	REC-003, REC-001
Q13	timeline	For how long were checkout payments failing?	REC-001, REC-003, REC-002
Q14	conflicting evidence	The API spec says card_token is still accepted. Why did 2.4.0 reject it when 2.3.1 accepted it?	DOC-002, WEB-001, DOC-004, REC-001
Q15	conflicting evidence	The initial report blames a credential problem. Is that still the best explanation?	DOC-005, TRN-002, DOC-002
Q16	conflicting evidence	The 2.4.0 release notes say there are no breaking changes. Is that correct?	DOC-004, REC-004, WEB-001
Q17	unanswerable	Which person approved the payment-service 2.4.0 production deploy?	TRN-003, TRN-001 (abstain)
Q18	unanswerable	How much revenue did ShopSphere lose during the incident?	TRN-003 (abstain)
Q19	unanswerable	Will setting allow_aliases: true in payment-service 2.4.0 fix the failures?	TRN-003, WEB-001 (abstain)
Q20	unanswerable	What request payload does the staging smoke test suite send?	TRN-001, TRN-003 (abstain)

Appendix B. True PDF page numbers

expected_sources.json gives pages 1-2 for these sections, but the real PDFs are 10-11 pages. The benchmark compares source IDs only, so this matters for citations, not for the scores.

Source	Heading	Actual page
DOC-001	3. Checkout Payment Flow	5
DOC-001	4. Retry Behaviour Between Services	7
DOC-002	4. Deprecated Fields	7
DOC-002	5. Error Codes	8
DOC-003	5. Rollback Considerations	7

Source	Heading	Actual page
DOC-004	2.1 Dependency Updates	4
DOC-004	4. Compatibility	5
DOC-005	3. Suspected Cause	4

Appendix C. Glossary

Term	Meaning
Chunk	One searchable passage with its location, labels and embedding
Embedding	768 numbers that capture what a passage means; similar meaning, nearby numbers
Vector index	MongoDB index over embeddings; $vectorSearch finds passages by meaning
BM25 / keyword index	MongoDB Atlas Search index over words; $search finds exact words
Reciprocal rank fusion	Merging ranked lists: each result scores 1 / (60 + rank) per list
Locator	Where a passage came from: page and section, html_id, cue_id, log_id, row_id...
Abstention	Saying the evidence does not answer the question
GridFS	MongoDB's way of storing whole files inside the database
Incident	Our 'project': a saved set of evidence; every search stays inside one incident